

How Many Capabilities Is Too Many? On the Scaling Limits of LLM-Based Malware Capability Classification

Anonymous Authors

Anonymous Institution

Abstract. Large language models (LLMs) are increasingly used to assist malware analysis. A key task is capability identification: mapping program artifacts to capability labels drawn from large candidate sets. This requires evaluating many labels, most of which are irrelevant for a given sample, raising a central question: how do LLM-based systems behave as the label space grows? We present a measurement study of label-space scaling in LLM-based malware capability classification using a controlled two-stage pipeline that fixes analysis evidence and varies only the number of candidate labels and the classification model. Across models, we observe a consistent pattern: as the label space increases, precision decreases while recall remains relatively stable. For example, precision drops from 0.71 to 0.50 as the label space grows from 40 to 198 labels, while false positives more than double. Because each false positive requires manual verification, we use false positives as a proxy for analyst effort. Increasing the label space leads to systematic over-prediction even with identical input, revealing a key limitation: LLM-based capability classification exhibits poor label-space scaling.

Keywords: Malware Analysis · Large Language Models · Multi-Label Classification · Measurement Study · Reverse Engineering

1 Introduction

Malware analysis aims to understand what malicious programs do in order to support tasks such as triage, incident response, and detection development [2, 21, 27]. Studies of expert analysts show that this process is inherently iterative and labor-intensive, requiring the interpretation of program artifacts through a combination of static and dynamic analysis [27]. Recently, industry systems and reports suggest that large language models (LLMs) can assist analysts by summarizing code and interacting with reverse engineering tools [17, 23].

A key step in this workflow is capability identification: mapping observed program artifacts to capability labels drawn from structured capability sets such as the Malware Behavior Catalog (MBC), which defines hundreds of fine-grained behaviors [13]. Here, a capability is a specific functionality of a program, represented as an MBC label. Automated systems must therefore evaluate many candidate labels, most of which are irrelevant for any given sample, creating a

challenging multi-label classification problem. However, unlike traditional classifiers that operate over a shared learned representation [28], LLM-based systems evaluate each label through separate language-based judgments over the same evidence. This design introduces a key failure mode: errors can accumulate as the number of candidate decisions increases, even when the input is unchanged. How these systems behave as the number of candidate labels grows remains unclear.

This raises a central question:

How does increasing the number of candidate capability labels affect LLM-based classification behavior?

In this paper, we study how LLM-based malware capability classification behaves as the candidate label space grows. We use a controlled two-stage pipeline that separates artifact extraction from classification, allowing us to hold the input evidence fixed while varying only the number of candidate labels and the classification model. This design isolates label-space scaling under controlled conditions and enables direct measurement of its effect on precision, recall, and false positives. Each false positive corresponds to a concrete analyst action (e.g., inspection, validation, or rule writing), making false positives a direct proxy for operational cost. A $2\times$ increase in false positives therefore directly translates to increased analyst workload.

Our results reveal a consistent pattern: as the label space increases, LLMs frequently over-predict unsupported capabilities, even when the input evidence is unchanged. For larger models, precision drops from 0.71 to 0.50 as the label space increases from 40 to 198 labels, while total false positives more than double ($281 \rightarrow 590$, a $2.1\times$ increase in verification burden), with little change in recall. This behavior indicates that LLM-based capability classification exhibits poor label-space scaling. Unlike traditional multi-label classifiers, labels are evaluated independently over shared context, allowing errors to accumulate as the number of candidate decisions increases.

We make the following contributions:

1. **Controlled experimental methodology.** We introduce a two-stage pipeline that separates artifact extraction from classification, enabling controlled evaluation across models and label sets.
2. **Measurement of label-space scaling effects.** We provide a systematic evaluation of how performance changes as the number of candidate labels increases, showing consistent degradation in precision.
3. **Empirical characterization of model behavior.** We show that larger label spaces are associated with systematic over-prediction, shifting errors toward false positives.
4. **Analysis of practical impact.** We quantify how increases in false positives translate to analyst verification effort, exceeding a $2\times$ increase as label space grows.

To promote future work, we open-source our evaluation data and experimental setup.¹

¹ Link redacted for blind review

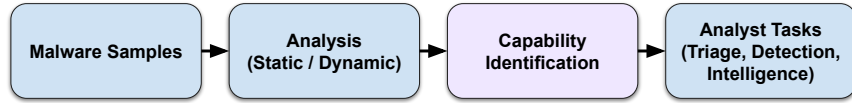


Fig. 1: Simplified malware analysis workflow. This work focuses on the capability identification step, where program artifacts are mapped to capability labels, and studies how LLM-based systems behave as the number of candidate labels increases.

Table 1: Preliminary Label-Space Scaling on a Mirai Sample

Label Set	Precision	Recall	False Positives
Observed Reference-Label Set (22)	1.00	0.86	0
Full Label Set (198)	0.53	0.86	17

2 Overview

Malware capability identification is one step within a broader analysis workflow (Figure 1). Analysts extract observable program artifacts from malware samples and map them to capability labels, supporting downstream tasks such as triage, incident response, detection development, and threat intelligence [2, 21, 27]. In this paper, we focus on the capability identification step and study how large language models (LLMs) perform this task as the number of candidate capability labels increases.

To motivate our study, we begin with a controlled preliminary experiment designed to isolate the effect of label-space size. We use a Mirai botnet sample with publicly available source code, allowing approximate validation of capabilities through manual inspection. The sample is compiled and analyzed using static analysis tools to produce a transcript of observable artifacts (e.g., functions and strings). This transcript is held constant across all experimental conditions so that any differences in performance can be attributed in this controlled setting to changes in the candidate label set rather than variation in input evidence. We use CAPA [21], an industry-standard open-source tool for mapping program artifacts to capability labels, to derive reference capabilities.

2.1 Preliminary Experiment: Label-Space Scaling on Mirai

We evaluate the same fixed evidence under two conditions: (1) a reference set containing only CAPA-derived capability labels present in the sample, and (2) the full CAPA-mapped MBC label set (198 labels), which introduces additional distractors. In both cases, the model evaluates each candidate label using only the provided evidence.

Table 1 shows that expanding the label space reduces precision while leaving recall unchanged. Under the reference set, the model achieves perfect precision

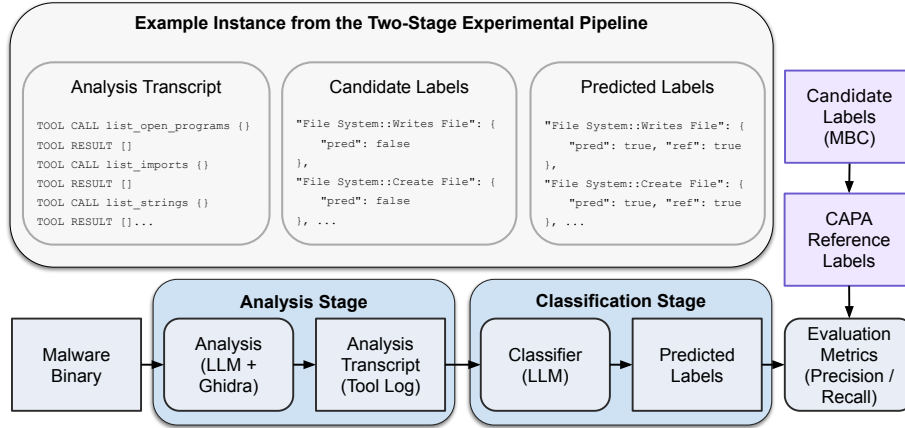


Fig. 2: Two-stage experimental pipeline. Static analysis produces a fixed transcript of observable artifacts, which a separate model uses to evaluate candidate capability labels. This separation fixes input evidence and isolates the effect of label-space size and model size on classification performance.

in this minimal candidate-set condition, with no false positives. When evaluated on the full label set using identical input evidence, precision drops to 0.53 due to 17 additional false positives, while recall remains at 0.86. Because the input evidence is held constant, this comparison isolates label-space size as the primary experimental variable associated with the increase in false positives.

To better understand the nature of these errors, we manually analyze the false positive predictions for this sample. We find that false positives fall into two categories: some are semantically related to the observed artifacts, reflecting mismatches in granularity (e.g., inferring HTTP communication from socket-level behavior), while others are unsupported by the evidence (e.g., reverse shell capabilities), indicating clear over-prediction. Both occur despite identical input, suggesting that increasing the number of candidate decisions is associated with systematic error accumulation. A detailed breakdown is provided in Appendix B.

This preliminary result suggests that increasing the label space can degrade precision without improving recall. While this experiment is limited to a single sample, it motivates a broader question: does this behavior generalize across malware families, model sizes, and label sets? We investigate this systematically in the following sections.

3 Design

Our goal is to isolate the effect of label-space size on capability classification, which requires controlling the analysis evidence provided to the classifiers. In

preliminary experiments, directly using raw strings or disassembly led to significant variability in input sizes across samples, causing context truncation and confounding label-scaling results. We therefore use a two-stage pipeline in which a bounded analysis stage produces fixed, bounded evidence for classification (Figure 2). This design avoids introducing uncontrolled variation in the evidence available to the classifier, ensuring that observed performance differences are primarily attributable to controlled changes in label-space size rather than variation in input representation.

For each malware binary, we generate a single analysis transcript and treat it as fixed input across all experimental conditions, ensuring that observed performance differences arise solely from differences in label-space size or classification model. We do not evaluate the analysis stage; it serves only to produce fixed, bounded evidence for classification.

3.1 Analysis Stage

In the first stage, an input binary undergoes static analysis to extract observable program artifacts, including imported libraries, strings, and decompiled functions.

Rather than relying on raw strings or disassembly, we use a tool-driven analysis process that selectively queries relevant program components under a fixed tool budget to bound transcript size and reduce variability in extracted evidence across samples. Each query and its result are recorded in a structured transcript, forming a bounded evidence trace for each sample.

The transcript contains only extracted artifacts and does not include capability predictions or candidate labels. This design ensures that the analysis stage remains independent of the classification task and prevents exposure to candidate labels during analysis, avoiding unbounded or sample-dependent input representations that would introduce uncontrolled variation in the evidence.

3.2 Classification Stage

In the second stage, a separate model evaluates a fixed set of candidate capability labels using only the analysis transcript as input. For each label, the model determines whether sufficient supporting evidence is present in the transcript.

The classifier has no access to the original binary or analysis tools. Its output is a structured JSON object containing one boolean prediction per label, enabling direct measurement of precision, recall, and false positives as the number of candidate labels increases.

Conceptually, this setting resembles binary relevance approaches in multi-label classification [28], where each label corresponds to a separate decision over the same evidence. However, unlike traditional classifiers that operate over a shared learned representation, the LLM evaluates each label through separate language-based judgments over the same transcript evidence, which may introduce dependencies through shared context.

4 Evaluation

We aim to answer the following research questions:

1. **How accurately can LLM-based systems classify malware capabilities?** We evaluate precision and recall across all samples with analysis transcripts held fixed (Section 4.2).
2. **How does performance scale with label-space size?** We vary the number of candidate capability labels and measure the impact on precision and recall (Section 4.3).
3. **What are the failure modes?** We analyze false positives and false negatives to identify systematic error patterns (Section 4.4).
4. **How does label-space scaling affect analyst verification effort?** We quantify how false positives grow with label-space size and the resulting verification burden (Section 4.5).

4.1 Experimental Setup

Environment Experiments are conducted on Ubuntu 22.04 with an Intel Xeon Silver 4310 CPU, 250 GB RAM, and an NVIDIA A30 GPU (24 GB VRAM). Model inference is performed locally using llama.cpp [7] with GPU acceleration. Static analysis is performed using headless Ghidra [14] via the Ghidra MCP server [3], and orchestration is implemented with AutoGen [26].

Dataset and Ground Truth Approximation We collect source code for Linux malware written in C from two public repositories [5, 24], including samples such as botnets, rootkits, and ransomware, and compile 58 Linux x86_64 ELF binaries using GCC 13.3.0 with `-O2` optimization and stripped symbols.

We derive reference capability labels using CAPA and restrict evaluation to Malware Behavior Catalog (MBC) capabilities [13]. We exclude MITRE ATT&CK techniques [22], as ATT&CK captures higher-level adversary behaviors while MBC focuses on program-level capabilities. CAPA outputs are treated as a reproducible reference baseline rather than ground truth, as static analysis cannot capture all behaviors and CAPA rules may be incomplete; our evaluation focuses on relative trends across label-space sizes rather than absolute correctness. The resulting label distribution is highly imbalanced with a long tail of rare capabilities (Figure 3). We report precision and recall rather than accuracy, as true negatives dominate in this setting; we micro-average across samples to summarize overall performance.

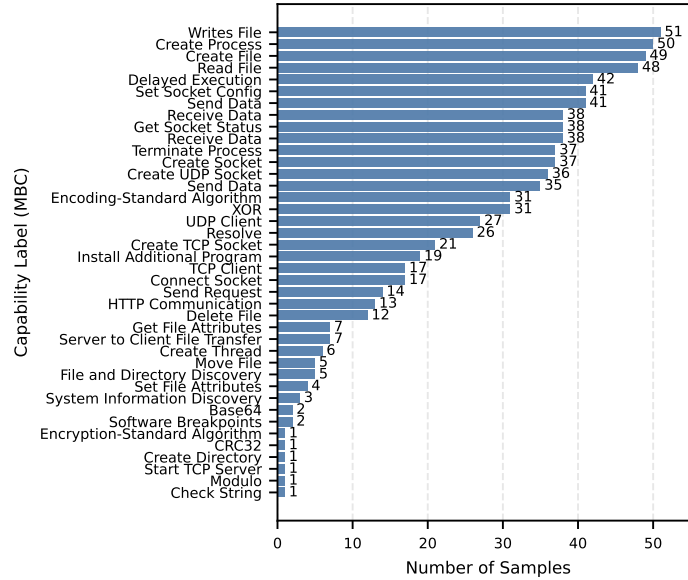
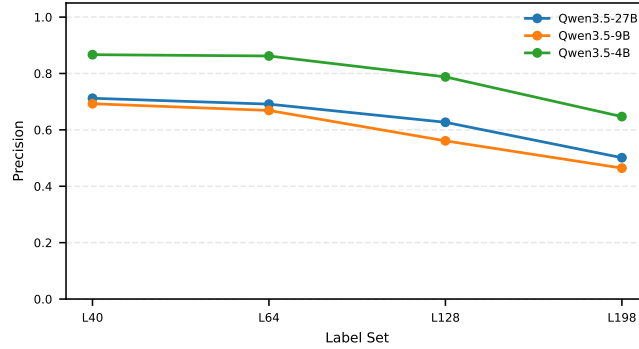


Fig. 3: Distribution of CAPA-derived MBC labels. The label space is highly imbalanced with a long tail, motivating the use of precision and recall over accuracy and helping explain why errors concentrate in common behaviors.

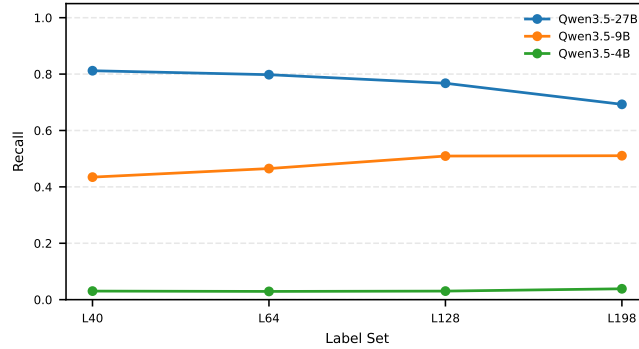
Label Subset Generation We evaluate label sets of sizes 40, 64, 128, and 198, where 198 corresponds to the full CAPA-mapped MBC label set. The smallest set (40) includes all observed reference labels, ensuring recall is always defined. Larger sets are constructed by adding distractor labels from the remaining MBC universe. All sets are nested supersets, allowing controlled scaling of candidate decisions while holding true labels

Models and Inference To avoid confounding cross-model differences, we use a single model family. We evaluate Qwen3.5-4B, 9B, and 27B [18] with deterministic decoding (temperature 0) and a 65K-token context window (prompts capped at 49K to avoid truncation). Analysis transcripts are generated once per sample using the largest model (27B) and reused to standardize input and isolate the effects of model size and label-space size. The analysis stage is limited to 10 tool calls per sample, as larger budgets showed little benefit.

Control Variables We use fixed prompts for both stages. The analysis stage does not receive candidate labels, preventing label leakage. The classification stage receives identical transcripts and label sets across models, ensuring that differences arise solely from model size and label-space size. Full prompts are provided in the appendix.



(a) Precision vs. label-space size.



(b) Recall vs. label-space size.

Fig. 5: Precision and recall as label space grows (L40–L198). Precision decreases across all models, while recall remains relatively stable, indicating more false positives without improved coverage.

poorly overall, with mean precision 0.07 and mean recall 0.03, indicating both low precision and low recall across samples.

The corresponding micro-averaged precision and recall on L198 are 0.50/0.69 for 27B, 0.46/0.51 for 9B, and 0.65/0.04 for 4B.

4.3 Scaling Trends

Figure 5 shows that precision decreases as the label space grows for all three models. From L40 to L198, precision drops from 0.71 to 0.50 for 27B, from 0.69 to 0.46 for 9B, and from 0.87 to 0.64 for 4B. Over the same range, recall remains relatively stable: 27B decreases from 0.81 to 0.69, 9B increases from 0.43 to 0.51, and 4B remains near zero at 0.03. Thus, increasing the number of candidate labels produces a consistent precision loss without a corresponding recall gain.

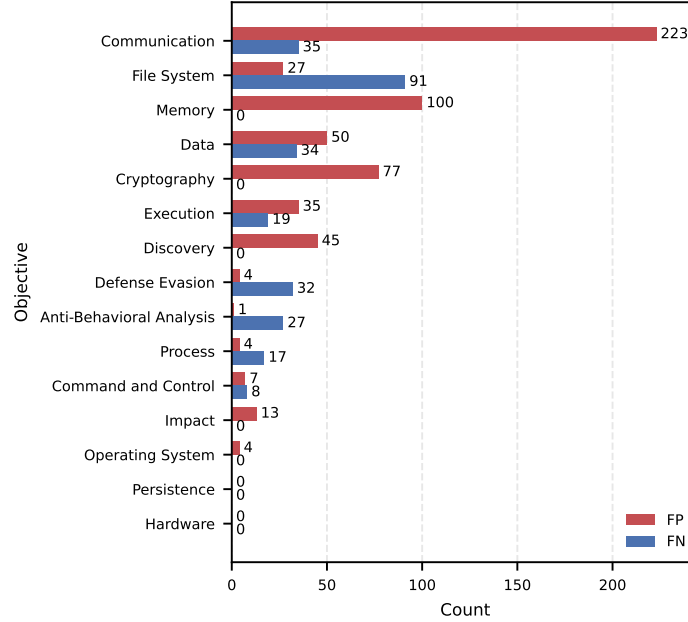


Fig. 6: Top objective-level false positives and false negatives for Qwen3.5-27B on L198. Errors are dominated by the Communication objective, with substantially higher counts than other categories.

4.4 Failure Modes

At the objective level (Figure 6), errors are highly concentrated in a small number of broad capability categories. Communication dominates both false positives and false negatives, followed by File System and Data. These objectives are common and semantically broad, allowing partial or ambiguous evidence to support multiple interpretations. In contrast, objectives such as Persistence, Hardware, and Operating System produce few or no predictions, indicating that rare or specialized capabilities are systematically under-represented. This concentration suggests that error accumulation is driven by high-frequency objectives where weak evidence can be reused across multiple related decisions.

At the behavior level (Figure 7), errors concentrate in a small number of behaviors. Socket Communication produces 153 false positives and 9 false negatives, indicating strong over-prediction. Other high-false-positive behaviors include Generate Pseudo-random Sequence (70), Interprocess Communication (66), and Allocate Memory (53). In contrast, Create File and Writes File are dominated by false negatives, with 41 and 36, respectively. These results highlight a divide between broadly defined behaviors that encourage overgeneralization and more specific behaviors that require precise evidence.

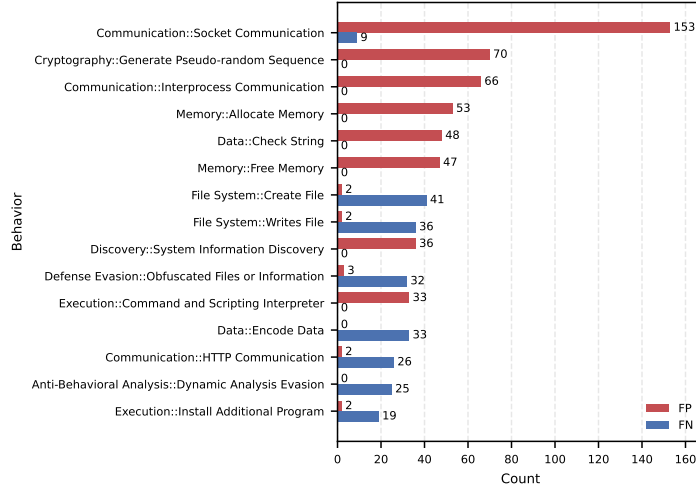


Fig. 7: Top behavior-level false positives and false negatives for Qwen3.5-27B on L198. Socket Communication dominates errors, with significantly higher false positives than other behaviors.

Method-level results (Figure 8) reveal three distinct failure patterns. First, several methods exhibit high false positive rates, including capabilities related to memory operations, data processing, and network communication. For example, operations such as memory allocation, string checking, and socket communication produce substantially more false positives than false negatives. These errors arise when partial or ambiguous evidence is reused to support multiple related capabilities, leading to correlated over-prediction.

Second, many file system and process-related methods are false-negative dominated. Operations such as Create File, Writes File, and Read File require precise and explicit evidence, and are therefore under-predicted when the analysis transcript does not contain sufficiently specific artifacts. This reflects a conservative bias for capabilities that require direct grounding in observable program behavior.

Third, several methods related to obfuscation, encoding, and delayed execution exhibit high false negatives with near-zero false positives. These capabilities often rely on implicit or weak signals that are not directly captured in the analysis transcript, leading to systematic under-detection.

Taken together, these results indicate that errors originate at the method level, where weak or partial evidence leads to both overgeneralization across semantically related capabilities and under-detection of capabilities requiring precise or implicit signals. Because all candidate labels are evaluated against a shared analysis context, such errors propagate across the label hierarchy, contributing to the observed increase in false positives as the label space grows.

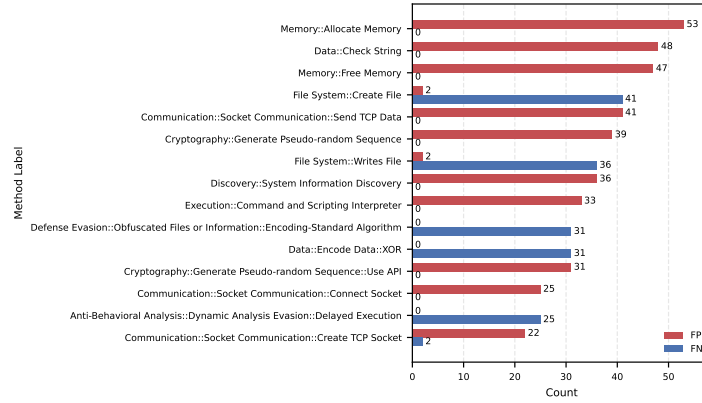


Fig. 8: Top method-level false positives and false negatives for Qwen3.5-27B on L198. Errors concentrate in a small number of methods, with multiple capabilities (e.g., memory and communication) exhibiting elevated false positives, while file system operations are dominated by false negatives.

Across models, error composition shifts with model size and label-space size. The 4B model is false-negative dominated, producing 830 false negatives and 4 false positives at L198 and 823 false negatives and 18 false positives at L198. The 9B model shifts from 484 false negatives and 165 false positives at L40 to 419 false negatives and 504 false positives at L198. The 27B model is false-positive-dominated throughout, increasing from 281 false positives and 161 false negatives at L40 to 590 false positives and 263 false negatives at L198.

4.5 Verification Burden

We use false positives as a proxy for analyst verification burden, as each predicted capability must be manually validated before use in downstream tasks. In this setting, the total number of false positives corresponds directly to the number of unnecessary verification actions performed by an analyst.

Figure 9 shows that false positives increase steadily with label-space size. For 27B, total false positives grow from 281 at L40 to 590 at L198. For 9B, false positives also increase with label-space size but remain below 27B at every setting. The 4B model remains consistently low in false positives due to under-prediction, but this comes at the cost of substantially reduced recall (Section 4.2).

Additionally, the increase in false positives is not offset by improved coverage. For 27B, recall decreases from 0.81 at L40 to 0.69 at L198 rather than increasing. This indicates that additional candidate labels primarily introduce unsupported predictions instead of recovering missed capabilities. As a result, expanding the label space increases analyst workload without providing corresponding benefits in capability identification.

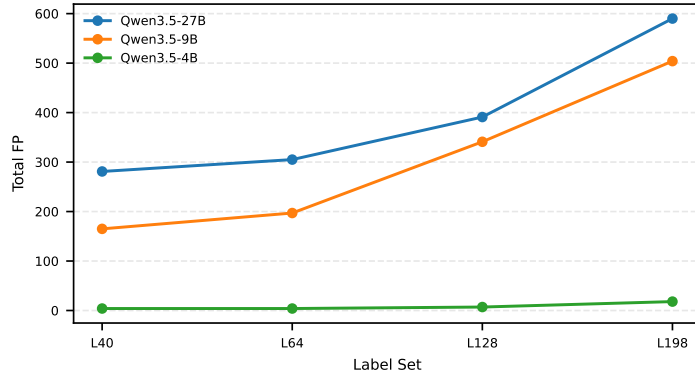


Fig. 9: Total false positives as the candidate label space grows. False positives increase monotonically with label-space size, leading to a more than 2 $\times$ increase in analyst verification burden for larger models.

Table 2: Verification burden for Qwen3.5-27B.

Label Set	Avg FP/sample	Total FP	Additional Total FP	Burden Multiplier
L40	4.84	281	0	1.00
L64	5.26	305	24	1.09
L128	6.74	391	110	1.39
L198	10.17	590	309	2.10

Table 2 quantifies this effect. For 27B, increasing the label space from L40 to L198 results in 309 additional false-positive predictions, corresponding to a 2.10 $\times$ increase in verification burden. On average, the number of false positives per sample more than doubles, from 4.84 to 10.17. This increase is monotonic across label-set sizes, indicating that each expansion of the candidate label space introduces additional verification cost.

To determine whether the observed increase in false positives is driven by a small number of outlier samples or is consistent across the dataset, we analyze the distribution of false positives per sample. Figure 10 shows that most samples cluster around moderate false-positive counts, but the distribution exhibits a long tail, with some samples producing significantly higher numbers of false positives. This indicates that while the increase in verification burden is broadly consistent across samples, certain binaries are particularly prone to over-prediction. As a result, larger label spaces not only increase average verification cost but also introduce variability in analyst workload, with some samples requiring more effort to validate.

This effect arises from the classification mechanism: because all candidate labels are evaluated independently against the same shared analysis context, weak or ambiguous evidence can be reused across multiple decisions. As the number of candidate labels increases, the number of opportunities for such reuse

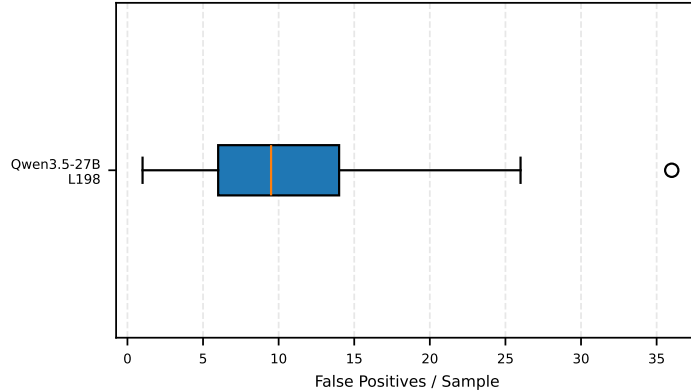


Fig. 10: Distribution of false positives per sample for Qwen3.5-27B at L198. While most samples cluster around moderate false-positive counts, variability is high, with some samples producing significantly higher numbers of false positives.

also increases, leading to systematic growth in false positives. This directly links the observed verification burden to the failure modes identified in Section 4.4.

From a practical perspective, this scaling behavior limits the usefulness of large label sets in analyst workflows. Even moderate increases in label-space size can substantially increase manual review effort, potentially outweighing any marginal gains in capability coverage. Without additional structure, such as candidate filtering or hierarchical evaluation, LLM-based capability classification systems may impose prohibitive verification costs as the number of candidate labels grows.

5 Discussion

We analyze the observed scaling behavior and identify key failure modes.

In this setting, increasing the size of the candidate label space leads to a systematic increase in false positives. A key contributing factor is that all candidate labels are evaluated against the same shared analysis context. Although labels are evaluated as separate decisions, those decisions are effectively coupled through shared context. As the number of candidate labels increases, opportunities for reuse also increase, contributing to error accumulation. In particular, ambiguous or weak evidence can be reused across multiple decisions, leading to false positives as the label space grows.

While the prompt length increases with the number of candidate labels, the observed degradation is not uniform: precision decreases significantly while recall remains relatively stable. This asymmetric effect is inconsistent with general context-length degradation and instead reflects error accumulation across independent label evaluations.

We observe that certain binaries are more prone to false positives due to ambiguous or broadly indicative signals (e.g., generic network activity), which can be reused across multiple candidate labels and lead to correlated over-prediction.

Manual inspection of model outputs supports this interpretation. We observe two common categories of false positives. First, semantically related errors arise when the model overgeneralizes from partial evidence to related capabilities. For example, evidence of HTTP communication may lead to predictions of multiple communication-related capabilities, even when only a subset is supported. Second, unsupported errors occur when the model predicts capabilities without clear grounding in the analysis transcript, suggesting that the model may infer functionality beyond what is directly supported by the observed artifacts.

These failure modes explain the observed performance trends. Recall remains relatively stable because true capability signals are still present and detectable in the shared context. In contrast, precision degrades as the label space grows, since the number of incorrect but weakly supported predictions increases. This behavior differs from traditional multi-label classifiers, where predictions are typically jointly calibrated. In our setting, independent evaluation over shared context allows errors to accumulate as the number of candidate decisions increases.

We also observe consistent differences across model sizes. Smaller models tend to under-predict, resulting in higher precision but low recall. Larger models are more likely to over-predict, increasing recall but also introducing more false positives. Mid-sized models exhibit a transition between these behaviors as the label space grows. Across all models, however, the same qualitative scaling trend holds: increasing the number of candidate labels leads to increased false positives without corresponding improvements in recall.

Additionally, increasing model size does not resolve the observed scaling behavior. While larger models improve recall, they also introduce additional false positives, leading to similar or worse precision as the label space grows. This suggests that the observed behavior arises from the classification mechanism rather than model size, and that scaling model size alone is unlikely to address poor label-space scaling.

5.1 Limitations

We outline key limitations that may affect absolute performance but do not alter the observed scaling trends.

Our evaluation uses CAPA-derived labels as a reproducible reference baseline rather than ground truth. While CAPA-derived labels are not complete, they provide a consistent reference across all experimental conditions. Because our study focuses on relative changes under identical inputs, any systematic bias in CAPA affects all conditions equally and therefore does not explain the observed scaling trends.

Our analysis is limited to static analysis of unpacked Linux binaries, which may not capture capabilities that require dynamic analysis to observe. Although restricting the dataset to Linux binaries reduces variability in tooling, it may

limit generalization to other platforms, particularly Windows malware. Consistent with this, we observe that CAPA produces a larger and more diverse set of capability labels for Windows binaries, likely reflecting differences in rule coverage. The dataset is also dominated by botnet samples, leading to an overrepresentation of network communication behaviors. While these factors may affect absolute error rates, the observed dependence on label-space size arises from the classification mechanism and is expected to persist across datasets and analysis settings.

5.2 Implications

These results highlight a limitation in this setting: LLM-based capability classification exhibits poor label-space scaling. Although labels are evaluated as separate decisions, those decisions are effectively coupled through shared context, leading to systematic error accumulation as the number of candidate labels increases. Without additional structure (e.g., candidate filtering or hierarchical labeling), such systems may not scale effectively to large label spaces in practice.

6 Related Work

6.1 LLM-Based Program Analysis and Agentic Systems

Recent work applies LLMs to program analysis and reverse engineering tasks, including binary summarization and static analysis [8, 12]. LLMs have also been explored for malware detection and classification [6, 16]. Agentic systems further augment LLMs with tool use to support complex cybersecurity workflows [25]. These approaches emphasize task-specific reasoning and end-to-end performance, often producing free-form outputs or coarse-grained predictions.

In contrast, our work studies LLMs in a structured multi-label classification setting, where decisions are made over a fixed set of candidate capabilities. Rather than improving task performance, we isolate label-space size as a controlled variable and evaluate how classification behavior changes as the number of candidate labels increases, a dimension that, to our knowledge, has not been directly examined in prior LLM-based program analysis or agentic systems.

6.2 Malware Capability Classification and Frameworks

Frameworks such as MITRE ATT&CK and the Malware Behavior Catalog (MBC) provide standardized representations of adversary capabilities [22, 13]. Widely-used tools such as CAPA [21] and CAPE [15] map program artifacts to these representations using expert-defined rules. Prior work has also explored predicting malware capabilities using dynamic analysis and symbolic execution, while traditional malware analysis has largely focused on detection and family classification [19, 20].

However, these approaches address different problem settings. Systems such as Forecast [2] rely on dynamic analysis and symbolic execution to predict capabilities from runtime memory images, whereas our work focuses on classifying capabilities from static analysis evidence within a fixed candidate set. Moreover, existing approaches either rely on manually engineered rules or operate over coarse-grained labels, and do not study automated, fine-grained capability classification using LLMs or how performance changes as the number of candidate capabilities grows.

6.3 Multi-Label and Extreme Multi-Label Classification

Multi-label classification introduces challenges such as label imbalance, large output spaces, and evaluation complexity, where precision and recall are preferred over accuracy due to sparsity [28]. In extreme multi-label settings, increasing the number of candidate labels often degrades precision due to accumulating false positives [4].

Separately, prior work has identified reliability issues in LLMs, including a tendency to produce unsupported or incorrect outputs [10] and performance degradation in long contexts where relevant signals compete with irrelevant information [11]. However, existing work does not examine how these challenges interact in LLM-based classification systems operating over large label spaces.

Our work bridges this gap by studying LLM-based classification through the lens of label-space scaling. We isolate label-space size as a controlled variable and show how increasing the number of candidate labels affects precision, recall, and analyst verification burden in a structured security setting.

7 Conclusion

In conclusion, LLM-based capability classification exhibits poor label-space scaling: increasing the number of candidate labels is associated with increased false positives without improving recall. This results in systematic over-prediction that grows with the label space, increasing false-positive review burden. Without additional structure (e.g., candidate filtering or hierarchical labeling), such systems do not scale with the number of candidate labels, limiting their practical deployment in real-world security workflows.

References

1. Aghakhani, H., Gritti, F., Mecca, F., Lindorfer, M., Ortolani, S., Balzarotti, D., Vigna, G., Kruegel, C.: When malware is packin’heat; limits of machine learning classifiers based on static analysis features. In: Network and Distributed Systems Security (NDSS) Symposium 2020 (2020)
2. Alrawi, O., Ike, M., Pruett, M., Kasturi, R.P., Barua, S., Hirani, T., Hill, B., Saltaformaggio, B.: Forecasting malware capabilities from cyber attack memory images. In: 30th USENIX Security Symposium

- (USENIX Security 21). pp. 3523–3540. USENIX Association (Aug 2021), <https://www.usenix.org/conference/usenixsecurity21/presentation/alrawi-forecasting>
3. Bethington, B.: Ghidra mcp server (2026), <https://github.com/bethington/ghidra-mcp>
 4. Dasgupta, A., Lamba, P., Kushwaha, A., Ravish, K., Katyan, S., Das, S., Kumar, P.: Review of extreme multilabel classification (2025), <https://arxiv.org/abs/2302.05971>
 5. Ding, F.: Iot malware (2017), <https://github.com/ifding/iot-malware>
 6. Feng, R., Chen, H., Wang, S., Monjurul Karim, M., Jiang, Q.: Llm-maldetect: A large language model-based method for android malware detection. *IEEE Access* **13**, 81347–81364 (2025). <https://doi.org/10.1109/ACCESS.2025.3565526>
 7. Gerganov, G., contributors: llama.cpp: Llm inference in c/c++ (2026), <https://github.com/ggml-org/llama.cpp>
 8. Jin, X., Larson, J., Yang, W., Lin, Z.: Binary code summarization: Benchmarking chatgpt/gpt-4 and other large language models (2023), <https://arxiv.org/abs/2312.09601>
 9. Kalapala, J.S., Zhang, L.: Packing induced bias in deep learning malware classifiers: A systematic experimental study. In: 2026 IEEE 5th International Conference on AI in Cybersecurity (ICAIC). pp. 1–6 (2026). <https://doi.org/10.1109/ICAIC67076.2026.11395794>
 10. Lin, S., Hilton, J., Evans, O.: Truthfulqa: Measuring how models mimic human falsehoods (2022), <https://arxiv.org/abs/2109.07958>
 11. Liu, N.F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., Liang, P.: Lost in the middle: How language models use long contexts. *Transactions of the association for computational linguistics* **12**, 157–173 (2024)
 12. Liu, P., Sun, C., Zheng, Y., Feng, X., Qin, C., Wang, Y., Xu, Z., Li, Z., Di, P., Jiang, Y., Sun, L.: Llm-powered static binary taint analysis. *ACM Trans. Softw. Eng. Methodol.* **34**(3) (Feb 2025). <https://doi.org/10.1145/3711816>
 13. MITRE: Malware Behavior Catalog, <https://github.com/MBCProject/mbc-markdown>
 14. National Security Agency: Ghidra (2026), <https://github.com/NationalSecurityAgency/ghidra>, version 12.0.3
 15. O’Reilly, K., Brukhovetsky, A.: CAPE: Malware Configuration And Payload Extraction, <https://github.com/kevoreilly/CAPEv2>
 16. Qian, X., Zheng, X., He, Y., Yang, S., Cavallaro, L.: Lamd: Context-driven android malware detection and classification with llms. In: 2025 IEEE Security and Privacy Workshops (SPW). pp. 126–136. IEEE (2025)
 17. Quintero, B.: Introducing virustotal code insight: Empowering threat analysis with generative ai (2023), <https://blog.virustotal.com/2023/04/introducing-virustotal-code-insight.html>, virusTotal Blog
 18. Qwen Team: Qwen3.5: Towards native multimodal agents (February 2026), <https://qwen.ai/blog?id=qwen3.5>
 19. Raff, E., Barker, J., Sylvester, J., Brandon, R., Catanzaro, B., Nicholas, C.: Malware detection by eating a whole exe (2017), <https://arxiv.org/abs/1710.09435>
 20. Ronen, R., Radu, M., Feuerstein, C., Yom-Tov, E., Ahmadi, M.: Microsoft malware classification challenge (2018), <https://arxiv.org/abs/1802.10135>
 21. The FLARE Team: capa, a tool to identify capabilities in programs and sandbox traces. (Jul 2020), <https://github.com/mandiant/capa>

22. The MITRE Corporation: Mitre att&ck (2026), <https://attack.mitre.org/>
23. Venere, G.: Using llms as a reverse engineering sidekick (2025), <https://blog.talosintelligence.com/using-llm-as-a-reverse-engineering-sidekick/>, cisco Talos Blog
24. vx-underground: Malwaresourcecode (2026), <https://github.com/vxunderground/MalwareSourceCode>
25. Wang, Z., Shi, T., He, J., Cai, M., Zhang, J., Song, D.: Cybergym: Evaluating ai agents' real-world cybersecurity capabilities at scale (2026), <https://arxiv.org/abs/2506.02548>
26. Wu, Q., Bansal, G., Zhang, J., Wu, Y., Li, B., Zhu, E., Jiang, L., Zhang, X., Zhang, S., Liu, J., Awadallah, A.H., White, R.W., Burger, D., Wang, C.: Autogen: Enabling next-gen llm applications via multi-agent conversation (2023), <https://arxiv.org/abs/2308.08155>
27. Yong Wong, M., Landen, M., Antonakakis, M., Blough, D.M., Redmiles, E.M., Ahamad, M.: An inside look into the practice of malware analysis. In: Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security. p. 3053–3069. CCS '21, Association for Computing Machinery, New York, NY, USA (2021). <https://doi.org/10.1145/3460120.3484759>, <https://doi.org/10.1145/3460120.3484759>
28. Zhang, M.L., Zhou, Z.H.: A review on multi-label learning algorithms. IEEE Transactions on Knowledge and Data Engineering **26**(8), 1819–1837 (2014). <https://doi.org/10.1109/TKDE.2013.39>

Appendix A Prompts

Prompt for Analysis Stage

Analyze the unknown binary using Ghidra.
Explore the program and collect concrete evidence of its behavior.
Continue until you have sufficient understanding or no new useful information is found.

System Message for Analysis Stage

You are an expert reverse engineer performing static malware analysis using Ghidra.
Use the available tools to analyze the unknown binary and collect concrete evidence of its behavior.

Constraints:

- Use only information obtained through tool interactions.
- Do not use outside knowledge or assume malware family identity.
- Do not classify capabilities or assign labels.
- Avoid redundant or repeated tool queries.

Exploration strategy:

- Prioritize identifying concrete artifacts such as:
 - APIs, libraries, and system calls
 - strings, constants, file paths, URLs, and command-line arguments
 - function behavior, control flow, and data transformations
- Continue querying tools while new meaningful evidence is discovered.
- Stop when additional queries yield little or no new useful information.

Prompt Template for Classification Stage

Task:

For each capability, determine whether sufficient evidence is present in the transcript.

JSON template:

<insert candidate label json>

Ghidra tool transcript:

<insert analysis transcript>

System Message for Classification Stage

You are a static malware capability classifier.
 You are given a Ghidra tool transcript for an unknown binary and a fixed set of candidate malware capabilities.
 Use only the provided transcript to determine whether each candidate capability is present in the program.

Constraints:

- Do not call tools.
- Do not use outside knowledge.
- Base decisions only on evidence from the transcript.
- Evaluate each capability independently.

Output requirements:

- Use the provided JSON template.
- Do not add or remove keys.
- Each value must be exactly "pred": true or "pred": false.
- true and false must be JSON booleans (not strings).
- Output only the final JSON object.

Appendix B Manual Analysis of False Positives

Predicted Capability	Source Evidence	Assessment
Communication::HTTP Communication::Send Request	Manual HTTP request construction; CAPA likely expects library-based evidence	Reasonable
Command and Control::C2 Communication::Server to Client File Transfer	No evidence of server-to-client file transfer in C2 loop	Unsupported
Impact::Remote Access::Reverse Shell	No callback shell; only outbound telnet communication	Unsupported
Discovery::File and Directory Discovery	Checks for Python on system	Reasonable
Discovery::System Information Discovery	Reads IP, CPU architecture, byte order, and version	Reasonable
Memory::Free Memory	Uses <code>free()</code> in multiple locations	Reasonable
Communication::HTTP Communication::Connect to Server	Manual connection logic rather than HTTP library calls	Reasonable
Execution::Command and Scripting Interpreter	Downloads and executes shell script	Reasonable

Predicted Capability	Source Evidence	Assessment
Cryptography::Generate Pseudo-random Sequence::Use API	Uses C standard library PRNG	Reasonable
Communication::Socket Communication::Send TCP Data	Function <code>sendTCP</code> suggests TCP transmission	Reasonable
Memory::Allocate Memory	Uses <code>malloc()</code> in multiple locations	Reasonable
Communication::HTTP Communication::Send Data	Function <code>sendHTTP</code> suggests HTTP transmission	Reasonable
Communication::HTTP Communication::Set Header	Constructs HTTP headers and randomizes user agents	Reasonable
Data::Check String	Uses <code>contains_string()</code> for string matching	Reasonable
Communication::HTTP Communication::Create Request	Builds HTTP request with <code>sprintf</code> into buffer	Reasonable
Cryptography::Generate Pseudo-random Sequence	Uses XOR shifting and CMWC PRNG	Reasonable
Communication::HTTP Communication::Download URL	No HTTP response handling; uses <code>wget</code> instead	Unsupported